

Dungeon Descent” – gra 2D w stylu roguelike

Autor: [Ruslan Stoliarov]

Kierunek: [Technologie Komputerowe]

Data: 16 lutego 2026

Spis treści

1. Wstęp i cel projektu
2. Wykorzystane narzędzia AI
3. Opis techniczny gry
4. Struktura kodu i najciekawsze fragmenty
5. Struktura repozytorium
6. Materiały źródłowe
7. Materiały edukacyjne
8. Podsumowanie

1. Wstęp i cel projektu

Celem projektu było stworzenie dwuwymiarowej gry w stylu **roguelike**. Gra zakłada stopniowe schodzenie w głąb lochów, gdzie z każdym poziomem pojawiają się coraz silniejsi przeciwnicy, a gracz po pokonaniu fali może wybrać ulepszenie swojej postaci. Projekt został zrealizowany w silniku **Unity 6** z wykorzystaniem języka **C#**. W trakcie prac nad grą wykorzystano szereg narzędzi sztucznej inteligencji do generowania kodu, assetów graficznych oraz dokumentacji.

2. Wykorzystane narzędzia AI

W projekcie zastosowano następujące narzędzia generatywnej sztucznej inteligencji:

Narzędzie	Model	Typ usługi	Zastosowanie
DeepSeek	DeepSeek Chat	Bezpłatny	Generowanie kodu C#, debugowanie, optymalizacja skryptów
PixelLab	–	Bezpłatny	Generowanie sprite'ów i tekstur tła
CraftPix	–	Bezpłatny	Źródło sprite'ów postaci (slime, wojownik)

Licencje: DeepSeek – udostępnia kod na licencji MIT, możliwość wykorzystania w projektach komercyjnych i edukacyjnych. PixelLab – zasoby na licencji Creative Commons (CC BY 4.0). CraftPix – darmowe assety z możliwością wykorzystania w grach (wymagane atribucje).

2.1. DeepSeek – generowanie i optymalizacja kodu C#

Przykład 1: Generowanie systemu ataku player

Pierwszym wyzwaniem było stworzenie systemu ataku, który uwzględniałby kierunek kursora, knockback wrogów oraz animacje. Zamiast pisać kod od zera, użyłem promptu:

"Napisz skrypt C# dla gracza w Unity 2D, który atakuje w kierunku kursora myszy. Atak ma zadawać obrażenia wrogom w promieniu, knockback, oraz uruchamiać animację. Użyj Coroutine do czasu trwania ataku."

Rezultat: DeepSeek wygenerował kompletny skrypt `PlayerAttack.cs` z korutyną `PerformAttack()` i metodą `CheckHit()`.

Odpowiedź AI – wygenerowany ulepszony kod:

```
csharp
Copy Download

public class PlayerAttack : MonoBehaviour
{
    // ... istniejące pola ...

    [Header("Attack Timing")]
    public float attackAnimationTime = 0.3f; // Czas trwania animacji
    private bool isAttacking = false;

    [Header("Knockback")]
    public float knockbackForce = 5f;

    IEnumerator PerformAttack()
    {
        isAttacking = true;

        // Zatrzymaj ruch gracza
        GetComponent<Rigidbody2D>().linearVelocity = Vector2.zero;

        // Animacja ataku
        animator.SetTrigger("Attack");

        // Czekaj na moment zadania obrażeń (np. w połowie animacji)
        yield return new WaitForSeconds(attackAnimationTime * 0.3f);
    }
}
```

```
void Update()
{
    if (Input.GetButtonDown("Fire1") && Time.time >= nextAttackTime && !isAttacking)
    {
        StartCoroutine(PerformAttack());
        nextAttackTime = Time.time + 1f / attackRate;
    }

    if (!isAttacking)
    {
        UpdateAttackPointPosition();
    }
}
}
```

Podczas testów zauważyłem, że wróg nie otrzymuje knockbacku we właściwym kierunku. Wkleiłem kod do DeepSeek z promptem:

"Knockback nie działa poprawnie – wróg odlatuje w złym kierunku. Oto kod: [wklejony kod]. Znajdź błąd."

AI szybko wskazało, że użyłem `transform.position` zamiast `rigidbody2D.position` i zasugerowało poprawkę.

```
csharp
Copy Download

[Header("Knockback")]
public float knockbackForce = 5f; // Siła odrzucenia
public float knockbackDuration = 0.2f; // Czas trwania odrzucenia

private bool isKnockedBack = false;

public void TakeDamage(int damage, Vector2 knockbackDirection)
{
    if (isKnockedBack) return;

    health -= damage;
    Debug.Log($"Przeciwnik otrzymał obrażenia! Pozostałe zdrowie: {health}");

    StartCoroutine(knockback(knockbackDirection));

    if (health <= 0)
    {
        Die();
    }
}

IEnumerator Knockback(Vector2 direction)
{
    isKnockedBack = true;

    if (rb != null)
    {
        rb.linearVelocity = Vector2.zero;
        rb.AddForce(direction * knockbackForce, ForceMode2D.Impulse);
    }

    yield return new WaitForSeconds(knockbackDuration);

    if (rb != null)
    {
        rb.linearVelocity = Vector2.zero;
    }
}
```

2.2. PixelLab – generowanie assetów graficznych

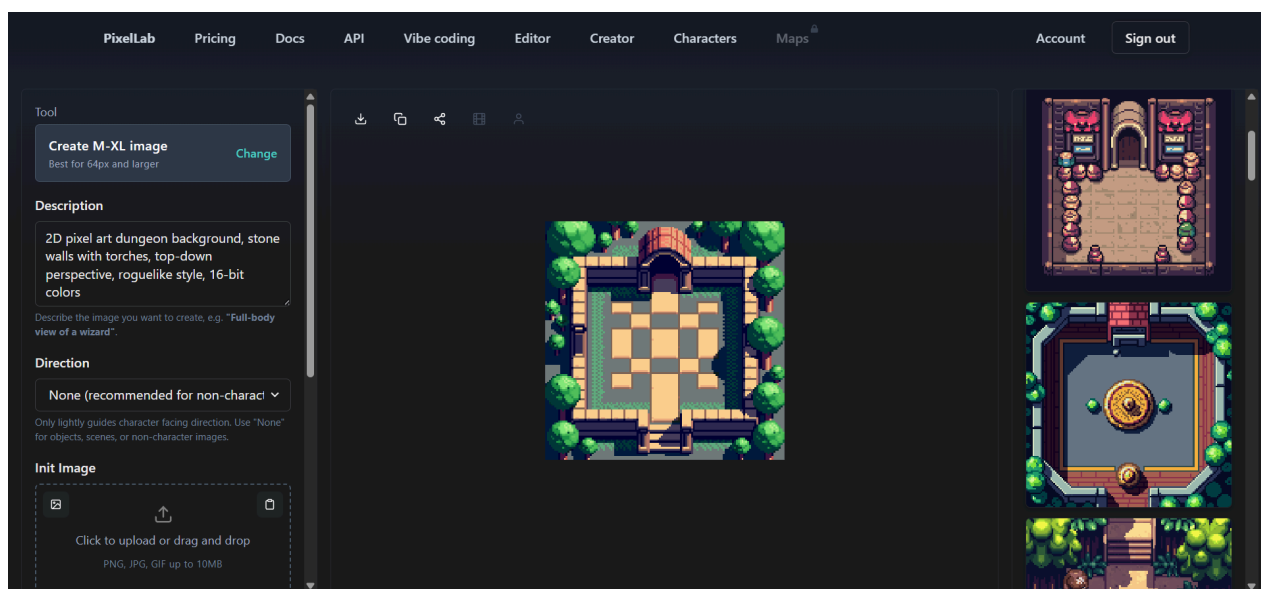
Do stworzenia tekstur tła wykorzystałem PixelLab, narzędzie AI do generowania grafik.

Przykład: Generowanie tekstury lochu

Prompt w PixelLab:

"2D pixel art dungeon background, stone walls with torches, top-down perspective, roguelike style, 16-bit colors"

AI wygenerowało 4 warianty tekstury, z których wybrałem najbardziej pasujący do stylistyki gry.



2.5. Prompt engineering – czego się nauczyłem?

Praca z AI wymagała nauki precyzyjnego formułowania promptów:

1. Kontekst jest kluczowy – zawsze podaję wersję Unity (6), typ gry (2D roguelike) i konkretny problem.
2. Iteracyjne doprecyzowanie – pierwszy prompt daje często 80% rozwiązania, kolejne dopracowują szczegóły.
3. Prośba o wyjaśnienia – gdy AI generuje kod, proszę o komentarze, aby zrozumieć logikę.

Przykład dobrego prompta:

"W Unity 6 2D, mam skrypt PlayerMovement oparty na Rigidbody2D. Chcę dodać dash w kierunku ruchu, z cooldownem 2 sekundy. Pokaż cały skrypt z komentarzami."

3. Opis techniczny gry

Gra została zbudowana w oparciu o architekturę **komponentową Unity**. Główne założenia: **Ruch postaci** – oparty na Rigidbody2D i Animator z blend tree dla 4 kierunków. **System fal wrogów** – dynamicznie generowane fale z rosnącym poziomem trudności. **Walka** – atak w kierunku kursora myszy, knockback wrogów, system zdrowia. **UI** – pasek zdrowia, licznik fal, menu pauzy i restartu. **Ulepszenia** – po każdej 3. fali gracz wybiera: zwiększenie obrażeń, prędkości lub pełne uzdrowienie.

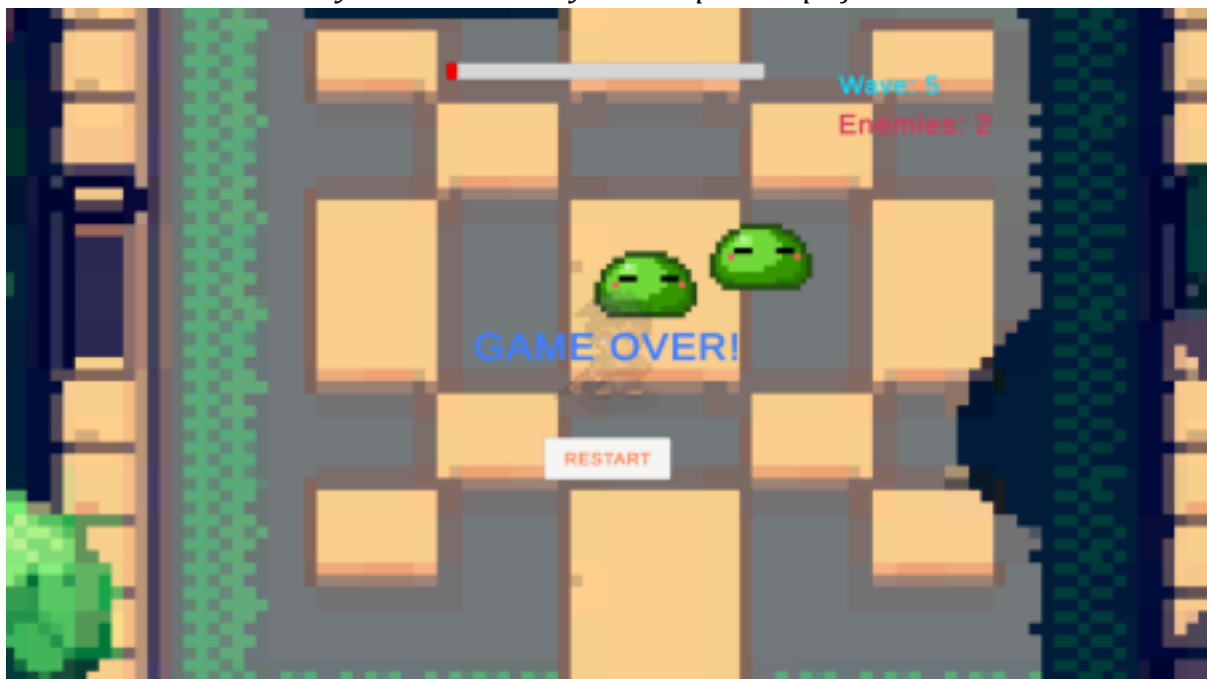
3.1. Zrzuty ekranu z gry



Rysunek 1. Rozgrywka – gracz atakuje slime'a



Rysunek 2. Menu wyboru ulepszenia po fali



Rysunek 3. Ekran Game Over

4. Struktura kodu i najciekawsze fragmenty

Kod został podzielony na logiczne moduły w katalogu Assets/Scripts. Poniżej omówiono najważniejsze z nich.

4.1. PlayerAttack.cs – system ataku

Kluczowym elementem jest metoda `PerformAttack()`, która zatrzymuje ruch gracza,

uruchamia animację ataku w kierunku kursora i zadaje obrażenia wrogom w zasięgu.

```
IEnumerator PerformAttack()
{
    isAttacking = true;
    rb.linearVelocity = Vector2.zero;
    animator.SetFloat("AttackDirectionX", lastAttackDirection.x);
    animator.SetFloat("AttackDirectionY", lastAttackDirection.y);
    animator.SetTrigger("IsAttacking");
    yield return new WaitForSeconds(0.1f);
    CheckHit();
    yield return new WaitForSeconds(attackAnimationTime);
    isAttacking = false;
}
```

Metoda CheckHit() wykorzystuje Physics2D.OverlapCircleAll do wykrycia wrogów w promieniu ataku.

```
void CheckHit()
{
    Collider2D[] hitEnemies = Physics2D.OverlapCircleAll(attackPoint.position, attackRadius, enemyLayers);
    foreach (Collider2D enemy in hitEnemies)
    {
        Enemy enemyScript = enemy.GetComponent<Enemy>();
        if (enemyScript != null)
        {
            Vector2 knockbackDir = (enemy.transform.position - transform.position).normalized;
            enemyScript.TakeDamage(Mathf.RoundToInt(damage), knockbackDir);
        }
    }
}
```

4.2. WaveManager.cs – dynamiczne fale

System fal korzysta z listy WaveSettings, która definiuje bazową liczbę wrogów, prędkość, zdrowie i kolor. Po każdej fali poziom trudności rośnie, a co 3 fale pojawia się menu ulepszeń.

```
if (upgradeManager != null && wavesSinceLastUpgrade >= wavesBetweenUpgrades)
{
    upgradeManager.ShowUpgradeMenu();
    wavesSinceLastUpgrade = 0;
}
```

4.3. UpgradeManager.cs – system ulepszeń

Skrypt zarządza wyborem ulepszeń i aktualizuje statystyki gracza.

```

public void OnDamageUpgrade()
{
    playerAttack.damage += 0.3f;
    HideUpgradeMenu();
}

public void OnSpeedUpgrade()
{
    playerMovement.moveSpeed += 0.2f;
    HideUpgradeMenu();
}

public void OnHealthUpgrade()
{
    playerHealth.Heal(playerHealth.maxHealth);
    HideUpgradeMenu();
}

```

4.4. PauseManager.cs – menu pauzy i startu

Skrypt kontroluje stan gry, wyświetlanie menu startowego i pauzy oraz reaguje na klawisz ESC.

```

void Update()
{
    if (gameStarted && Input.GetKeyDown(KeyCode.Escape))
    {
        if (isPaused)
            ResumeGame();
        else
            PauseGame();
    }
}

```

5. Struktura repozytorium

Repozytorium projektu zorganizowane jest w następujący sposób:


```

DungeonDescent/
├── Assets/
│   ├── Animations/           # Pliki animacji (.anim) i kontrolery (Animator Controller)
│   │   ├── Player/           # Animacje gracza (walk, attack, idle)
│   │   └── Enemy/             # Animacje wrogów
│   ├── Art/                   # Grafiki i sprite'y
│   │   ├── Backgrounds/      # Tła poziomów
│   │   ├── Sprites/          # Sprite'y postaci i obiektów
│   │   │   ├── Player/       # Sprite'y gracza
│   │   │   └── Slime/         # Sprite'y wrogów
│   │   └── Fonts/             # Czcionki
│   ├── Prefabs/               # Prefabry (gotowe obiekty do wielokrotnego użycia)
│   │   ├── Enemy.prefab
│   │   └── Player.prefab
│   ├── Scenes/                # Sceny gry
│   │   └── TestLevel.unity
│   └── Scripts/                # Kod źródłowy
│       ├── CameraFollow.cs
│       ├── Enemy.cs
│       ├── EnemyDeathHandler.cs
│       ├── GameManager.cs
│       ├── PauseManager.cs
│       ├── PlayerAttack.cs
│       ├── PlayerHealth.cs
│       ├── PlayerMovement.cs
│       ├── UpgradeManager.cs
│       └── WaveManager.cs
├── ProjectSettings/           # Ustawienia projektu Unity
└── README.md                  # Opis projektu

```

6. Materiały źródłowe

W projekcie wykorzystano następujące zasoby zewnętrzne:

Zasób Źródło Zastosowanie

Sprite'y slime'a CraftPix.net Wróg w grze

Sprite'y wojownika CraftPix.net Gracz

Tekstury tła PixelLab Tła poziomów Kod źródłowy DeepSeek Generowanie skryptów

C# Dokumentacja Unity docs.unity.com Rozwiązywanie problemów

Linki: CraftPix.net, PixelLab, DeepSeek

7. Materiały edukacyjne

Przy realizacji projektu korzystano z następujących materiałów: **Dokumentacja Unity** – docs.unity.com, **Kursy YouTube**: "Unity 2D Roguelike Tutorial" – Brackeys, "How to make a 2D game in Unity" – Code Monkey, **Stack Overflow** – rozwiązywanie problemów z kodem, **Forum Unity** – dyskusje na temat animacji i fizyki 2D, **ChatGPT / DeepSeek** – pomoc w generowaniu kodu i debugowaniu.

8. Podsumowanie

Projekt „Dungeon Descent” został zrealizowany zgodnie z założeniami. Udało się osiągnąć: działający system walki i fal wrogów, dynamiczny system ulepszeń, responsywne UI, płynne animacje postaci, menu pauzy i restartu. Wykorzystanie sztucznej inteligencji znacząco przyspieszyło proces tworzenia kodu oraz generowania assetów graficznych. Projekt pokazuje, jak nowoczesne narzędzia AI mogą wspierać proces tworzenia gier komputerowych.